Propositional Logic

Code:

```
def tt_entails(kb_symbols, kb_eval_fn, query_eval_fn):
    """
    TT-Check-All: Recursively evaluates whether KB entails Query.

    :param kb_symbols: List of symbol strings (e.g., ['P', 'Q'])
    :param kb_eval_fn: Lambda function to evaluate KB given a model dict
    :param query_eval_fn: Lambda function to evaluate Query given a model dict
    :return: Boolean (True if KB |= Query, False otherwise)
    """
    def tt_check_all(symbols, model):
        if not symbols:
            if kb_eval_fn(model):
                return query_eval_fn(model)
            else:
                return True
        else:
            p = symbols[0]
            rest = symbols[1:]
            model_true = model.copy()
            model_true[p] = True
            model_false = model.copy()
            model_false[p] = False

            return (tt_check_all(rest, model_true) and
                    tt_check_all(rest, model_false))

    return tt_check_all(kb_symbols, {})

symbols = ['P', 'Q', 'R']

kb_formula = lambda m: (not m['Q'] or m['P']) and (not m['P'] or not m['Q']) and (m['Q']
or m['R'])

query_ii  = lambda m: m['R']
query_iii = lambda m: not m['R'] or m['P']
query_iv  = lambda m: not m['Q'] or m['R']

print("ii)  Does KB entail R?      ->", tt_entails(symbols, kb_formula, query_ii))
print("iii) Does KB entail R -> P? ->", tt_entails(symbols, kb_formula, query_iii))
print("iv)  Does KB entail Q -> R? ->", tt_entails(symbols, kb_formula, query_iv))

import itertools

def print_truth_table(query_name, query_fn):
    symbols = ['P', 'Q', 'R']
    headers = ['P', 'Q', 'R', 'Q -> P', 'P -> ~Q', 'Q v R', 'KB', query_name]

    print(f"=== Truth Table checking KB |= {query_name} ===")
    header_str = " | ".join(f"{h:^8}" for h in headers)
```

```
    print(header_str)
    print("-" * len(header_str))

    # Iterate over all 8 truth assignments for (P, Q, R)
    for values in itertools.product([False, True], repeat=3):
        m = dict(zip(symbols, values))

        # Evaluate each sentence in KB
        s1 = not m['Q'] or m['P']        # Q -> P
        s2 = not m['P'] or not m['Q']    # P -> ~Q
        s3 = m['Q'] or m['R']            # Q v R

        kb = s1 and s2 and s3
        q_val = query_fn(m)

        row = [
            str(m['P']).lower(),
            str(m['Q']).lower(),
            str(m['R']).lower(),
            str(s1).lower(),
            str(s2).lower(),
            str(s3).lower(),
            str(kb).lower(),
            str(q_val).lower()
        ]

        row_str = " | ".join(f"{v:^8}" for v in row)
        if kb:
            row_str += "  <-- KB Model"
        print(row_str)
    print("\n")

# Questions from the image:
# ii) KB |= R
print_truth_table("R", lambda m: m['R'])

# iii) KB |= R -> P
print_truth_table("R -> P", lambda m: not m['R'] or m['P'])

# iv) KB |= Q -> R
print_truth_table("Q -> R", lambda m: not m['Q'] or m['R'])
```

Output:

Consider a knowledge base KB that contains the following propositional logic sentences:

$Q \rightarrow P$

$P \rightarrow \neg Q$

QVR

i) Construct a truth table that shows the truth value of each sentence in KB and indicate the models in which the KB is true.

ii) Does KB entail R?

iii) Does KB entail $R \rightarrow P$?

iv) Does KB entail $Q \rightarrow R$?

```
PS C:\Users\Administrator\Desktop\1BM24CS316> & 'C:\Python314\python.exe' 'c:/Users/Administrator/Desktop/1BM24CS316/PropositionalLogic.py'
ii)  Does KB entail R?      -> True
iii) Does KB entail R -> P? -> False
iv)  Does KB entail Q -> R? -> True
=== Truth Table checking KB |= R ===
   P    |    Q    |    R    | Q -> P  | P -> ~Q | Q v R  |    KB    |    R
-------------------------------------------------------------------------------
 false  |  false  |  false  |  true   |  true   | false  |  false   |  false
 false  |  false  |   true  |  true   |  true   |  true  |   true   |   true    <-- KB Model
 false  |   true  |  false  |  false  |  true   |  true  |  false   |  false
 false  |   true  |   true  |  false  |  true   |  true  |  false   |   true
  true  |  false  |  false  |  true   |  true   | false  |  false   |  false
  true  |  false  |   true  |  true   |  true   |  true  |   true   |   true    <-- KB Model
  true  |   true  |  false  |  true   | false   |  true  |  false   |  false
  true  |   true  |   true  |  true   | false   |  true  |  false   |   true


=== Truth Table checking KB |= R -> P ===
   P    |    Q    |    R    | Q -> P  | P -> ~Q | Q v R  |    KB    | R -> P
-------------------------------------------------------------------------------
 false  |  false  |  false  |  true   |  true   | false  |  false   |   true
 false  |  false  |   true  |  true   |  true   |  true  |   true   |  false    <-- KB Model
 false  |   true  |  false  |  false  |  true   |  true  |  false   |   true
 false  |   true  |   true  |  false  |  true   |  true  |  false   |  false
  true  |  false  |  false  |  true   |  true   | false  |  false   |   true
  true  |  false  |   true  |  true   |  true   |  true  |   true   |   true    <-- KB Model
  true  |   true  |  false  |  true   | false   |  true  |  false   |   true
  true  |   true  |   true  |  true   | false   |  true  |  false   |   true


=== Truth Table checking KB |= Q -> R ===
   P    |    Q    |    R    | Q -> P  | P -> ~Q | Q v R  |    KB    | Q -> R
-------------------------------------------------------------------------------
 false  |  false  |  false  |  true   |  true   | false  |  false   |   true
 false  |  false  |   true  |  true   |  true   |  true  |   true   |   true    <-- KB Model
 false  |   true  |  false  |  false  |  true   |  true  |  false   |  false
 false  |   true  |   true  |  false  |  true   |  true  |  false   |   true
  true  |  false  |  false  |  true   |  true   | false  |  false   |   true
  true  |  false  |   true  |  true   |  true   |  true  |   true   |   true    <-- KB Model
  true  |   true  |  false  |  true   | false   |  true  |  false   |  false
  true  |   true  |   true  |  true   | false   |  true  |  false   |   true
```